

MASTER OF INFORMATION TECHNOLOGY (MIT)
MIT 802: Introduction to Database

ASSIGNMENT

SUBJECT: Casual Case Study of a Cybersecurity Business Process

TOPIC:

**A Case Study of a Cybersecurity Incident Response and Threat Monitoring
System in a Security Operations Center (SOC)**

MATRIC NUMBER:

120505049

INSTITUTION:

University of Lagos

COURSE LECTURER:

Dr. Sennaike, Oladipupo A.

DATE OF SUBMISSION:

Saturday, May 30, 2026

Introduction

In today's digital environment, organizations are increasingly exposed to a wide range of cybersecurity threats such as malware attacks, phishing campaigns, unauthorized access attempts, and data breaches. As businesses continue to rely heavily on networked systems and cloud-based infrastructure, the need for an effective cybersecurity monitoring and incident response mechanisms has become critical.

A Security Operations Center (SOC) plays a central role in ensuring the confidentiality, integrity, and availability of every organizational information system and business is in place. It is responsible for continuously monitoring security events, detecting suspicious activities, analyzing potential threats, and coordinating timely responses to security incidents. To support these operations effectively, a well-structured database system is required to manage large volumes of security-related data, including assets, vulnerabilities, alerts, incidents, and remediation actions.

This case study focuses on the design and implementation of a Cybersecurity Incident Response and Threat Monitoring System within a SOC environment. The system is intended to model real-world cybersecurity operations by capturing how security analysts detect, investigate, and respond to incidents affecting organizational assets. It also demonstrates how relational database design principles can be applied to support efficient data storage, retrieval, and reporting in a cybersecurity context.

The objective of this work is to develop a structured database system that supports incident tracking, asset management, vulnerability monitoring, and security reporting. Through this, the study illustrates how a relational database can improve incident response efficiency, enhance situational awareness, and strengthen overall organizational security posture.

Description of the Business Process & Data Requirements

Just like the title our focus here is Cybersecurity Incident Response and Threat Monitoring System. The Business Description would involve a cybersecurity operation center (SOC) monitoring an organization networks, systems, and digital assets in order to detect, analyze, and respond to cybersecurity threats and incidents.

The organization requires a database system that:

- stores company assets.
- tracks vulnerabilities.
- records security incidents.
- assigns analysts to incidents.

- monitors alerts generated by security tools.
- and tracks remediation activities.

Note: The system will improve incident response time, accountability, asset visibility, and security reporting.

Data Requirements (Entities)

Entity	Description
Organization	Companies using the SOC services
Asset	Servers, laptops, firewalls, endpoints
Analyst	Cybersecurity analysts handling incidents
Incident	Security events such as malware or phishing
Alert	SIEM or IDS alerts
Vulnerability	Security weaknesses discovered
Remediation	Actions taken to fix incidents
Report	Final incident documentation

Conceptual Model (Entity Relationship Diagram)

The conceptual model represents the high-level structure of the Cybersecurity Incident Response and Threat Monitoring System. The Entity Relationship (ER) model was developed to identify the major entities involved in the system, their attributes, and the relationships that exist between them.

The purpose of the conceptual model is to provide a clear representation of how cybersecurity data is organized and how different components within the Security Operations Center (SOC) interact with one another.

The major entities identified in the system include Organization, Asset, Analyst, Incident, Alert, Vulnerability, Remediation, and Report.

Relationships Between Entities

The following relationships were identified within the system:

Relationship	Cardinality	Description
Organization → Asset	One-to-Many (1:M)	One organization can own multiple assets
Asset → Incident	One-to-Many (1:M)	One asset can experience multiple security incidents

Relationship	Cardinality	Description
Analyst → Incident	One-to-Many (1:M)	One analyst can handle multiple incidents
Incident → Alert	One-to-Many (1:M)	One incident can generate multiple alerts
Incident → Remediation	One-to-Many (1:M)	One incident can require multiple remediation actions
Incident → Report	One-to-One (1:1)	Each incident produces one final report
Asset → Vulnerability	One-to-Many (1:M)	One asset may contain multiple vulnerabilities

Entities and Attributes

Organization

- Org_ID (Primary Key)
- Org_Name
- Industry
- Contact_Email

Asset

- Asset_ID (Primary Key)
- Org_ID (Foreign Key)
- Asset_Name
- Asset_Type
- IP_Address
- Operating_System

Analyst

- Analyst_ID (Primary Key)
- Full_Name
- Email

- Certification
- Role

Incident

- Incident_ID (Primary Key)
- Asset_ID (Foreign Key)
- Analyst_ID (Foreign Key)
- Incident_Type
- Severity
- Status
- Detection_Date

Alert

- Alert_ID (Primary Key)
- Incident_ID (Foreign Key)
- Alert_Source
- Alert_Level
- Timestamp

Vulnerability

- Vulnerability_ID (Primary Key)
- Asset_ID (Foreign Key)
- CVE_Code
- Severity
- Discovery_Date

Remediation

- Remediation_ID (Primary Key)
- Incident_ID (Foreign Key)
- Action_Taken
- Action_Date

- Status

Report

- Report_ID (Primary Key)
- Incident_ID (Foreign Key)
- Report_Summary
- Generated_Date

Entity Relationship Diagram (ERD)

The figures below illustrate the Entity Relationship Diagram (ERD) for the Cybersecurity Incident Response and Threat Monitoring System.

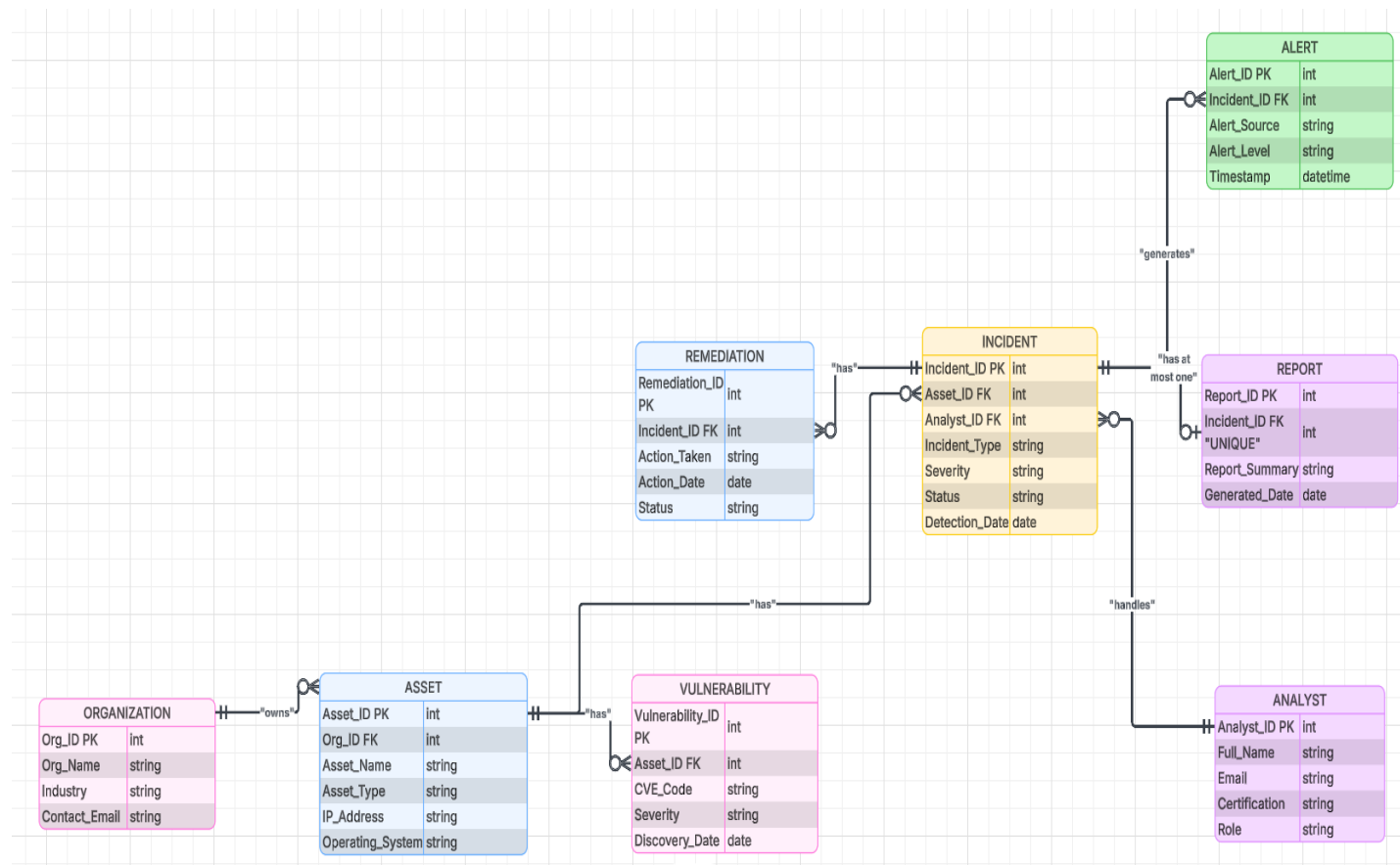


Figure 1: Entity Relationship Diagram for the Cybersecurity Incident Response and Threat Monitoring System

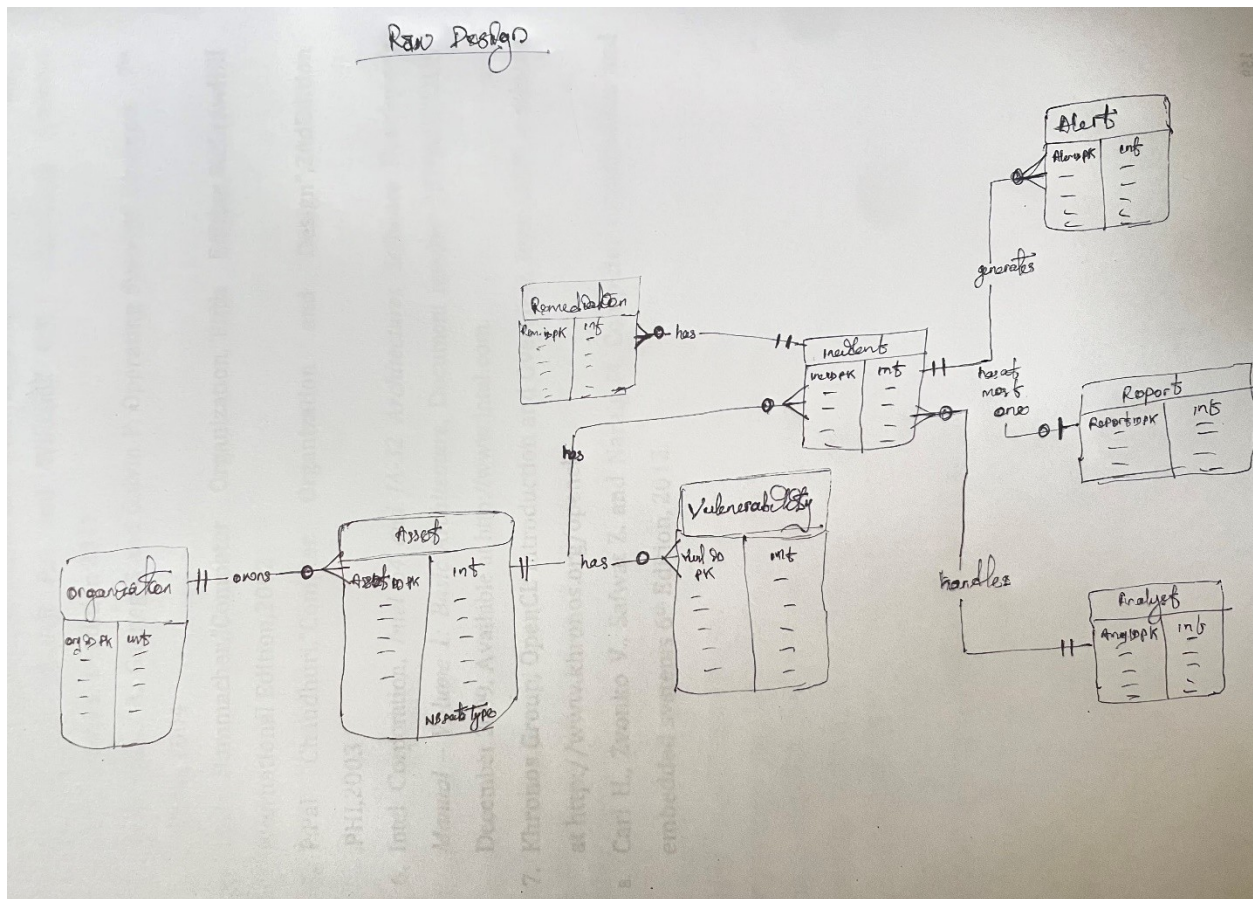


Figure 2: Entity Relationship Diagram for the Cybersecurity Incident Response and Threat Monitoring System raw design concept

Logical Model (Relations)

ORGANIZATION(

Org_ID PK,

Org_Name,

Industry,

Contact_Email

)

ASSET(

Asset_ID PK,

Org_ID FK,
Asset_Name,
Asset_Type,
IP_Address,
Operating_System
)

ANALYST(
Analyst_ID PK,
Full_Name,
Email,
Certification,
Role
)

INCIDENT(
Incident_ID PK,
Asset_ID FK,
Analyst_ID FK,
Incident_Type,
Severity,
Status,
Detection_Date
)

ALERT(
Alert_ID PK,

Incident_ID FK,
Alert_Source,
Alert_Level,
Timestamp
)

VULNERABILITY(
Vulnerability_ID PK,
Asset_ID FK,
CVE_Code,
Severity,
Discovery_Date
)

REMEDIATION(
Remediation_ID PK,
Incident_ID FK,
Action_Taken,
Action_Date,
Status
)

REPORT(
Report_ID PK,
Incident_ID FK,
Report_Summary,
Generated_Date

)

Physical Model

Attribute	Data Type
IDs	INT
Names	VARCHAR(100)
Emails	VARCHAR(150)
Severity	VARCHAR(20)
Dates	DATE
Reports	TEXT

DDL (Create Tables)

```
CREATE TABLE Organization (  
    Org_ID INT PRIMARY KEY,  
    Org_Name VARCHAR(100),  
    Industry VARCHAR(100),  
    Contact_Email VARCHAR(150)  
);
```

```
CREATE TABLE Asset (  
    Asset_ID INT PRIMARY KEY,  
    Org_ID INT,  
    Asset_Name VARCHAR(100),  
    Asset_Type VARCHAR(50),  
    IP_Address VARCHAR(50),  
    Operating_System VARCHAR(100),  
    FOREIGN KEY (Org_ID) REFERENCES Organization(Org_ID)  
);
```

```
CREATE TABLE Analyst (  
    Analyst_ID INT PRIMARY KEY,  
    Full_Name VARCHAR(100),  
    Email VARCHAR(150),  
    Certification VARCHAR(100),  
    Role VARCHAR(50)  
);
```

```
CREATE TABLE Incident (  
    Incident_ID INT PRIMARY KEY,  
    Asset_ID INT,  
    Analyst_ID INT,  
    Incident_Type VARCHAR(100),  
    Severity VARCHAR(20),  
    Status VARCHAR(50),  
    Detection_Date DATE,  
    FOREIGN KEY (Asset_ID) REFERENCES Asset(Asset_ID),  
    FOREIGN KEY (Analyst_ID) REFERENCES Analyst(Analyst_ID)  
);
```

```
CREATE TABLE Alert (  
    Alert_ID INT PRIMARY KEY,  
    Incident_ID INT,  
    Alert_Source VARCHAR(100),  
    Alert_Level VARCHAR(20),  
    Timestamp DATETIME,
```

```
FOREIGN KEY (Incident_ID) REFERENCES Incident(Incident_ID)
);
```

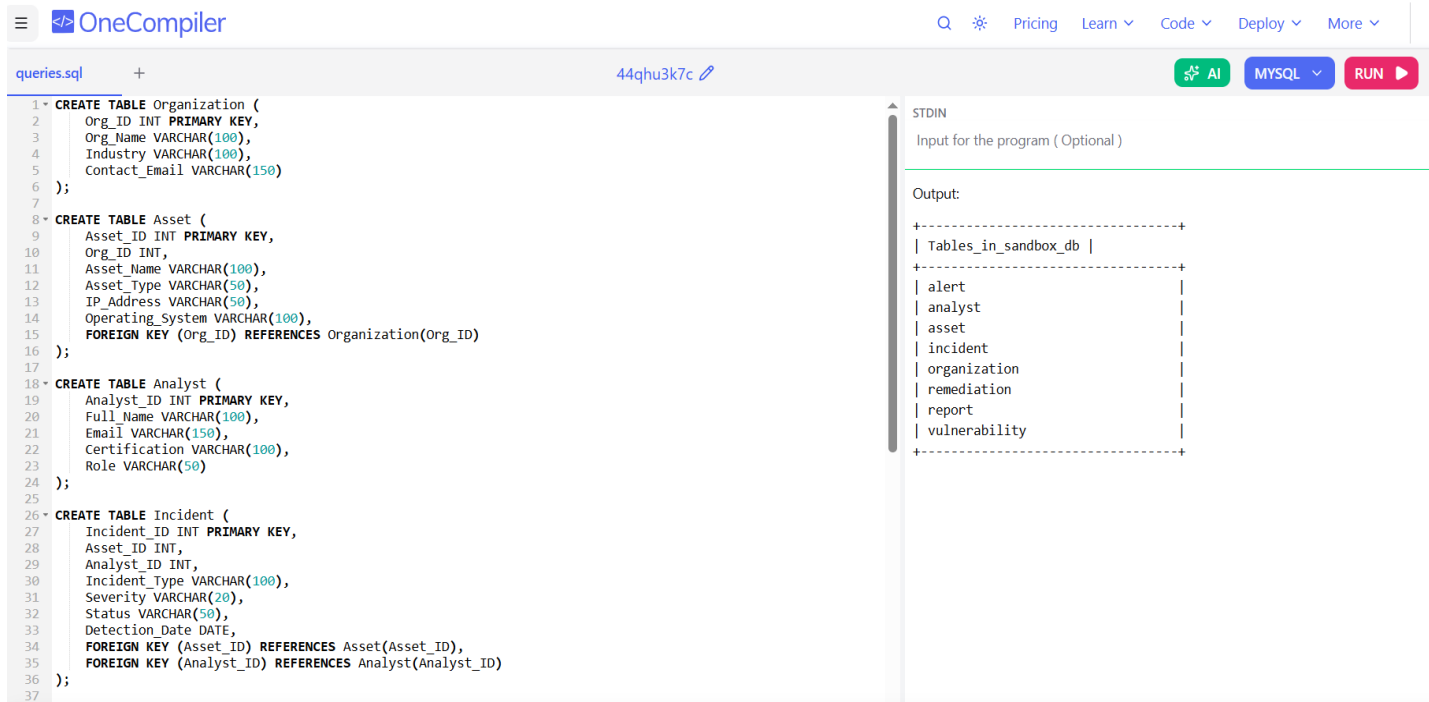
```
CREATE TABLE Vulnerability (
    Vulnerability_ID INT PRIMARY KEY,
    Asset_ID INT,
    CVE_Code VARCHAR(50),
    Severity VARCHAR(20),
    Discovery_Date DATE,
    FOREIGN KEY (Asset_ID) REFERENCES Asset(Asset_ID)
);
```

```
CREATE TABLE Remediation (
    Remediation_ID INT PRIMARY KEY,
    Incident_ID INT,
    Action_Taken TEXT,
    Action_Date DATE,
    Status VARCHAR(50),
    FOREIGN KEY (Incident_ID) REFERENCES Incident(Incident_ID)
);
```

```
CREATE TABLE Report (
    Report_ID INT PRIMARY KEY,
    Incident_ID INT UNIQUE,
    Report_Summary TEXT,
    Generated_Date DATE,
    FOREIGN KEY (Incident_ID) REFERENCES Incident(Incident_ID)
```

);

SHOW TABLES;



The screenshot shows the OneCompiler web interface with a MySQL query executed. The query creates four tables: Organization, Asset, Analyst, and Incident. The output shows the list of tables created in the database.

```
1 CREATE TABLE Organization (
2   Org_ID INT PRIMARY KEY,
3   Org_Name VARCHAR(100),
4   Industry VARCHAR(100),
5   Contact_Email VARCHAR(150)
6 );
7
8 CREATE TABLE Asset (
9   Asset_ID INT PRIMARY KEY,
10  Org_ID INT,
11  Asset_Name VARCHAR(100),
12  Asset_Type VARCHAR(50),
13  IP_Address VARCHAR(50),
14  Operating_System VARCHAR(100),
15  FOREIGN KEY (Org_ID) REFERENCES Organization(Org_ID)
16 );
17
18 CREATE TABLE Analyst (
19   Analyst_ID INT PRIMARY KEY,
20   Full_Name VARCHAR(100),
21   Email VARCHAR(150),
22   Certification VARCHAR(100),
23   Role VARCHAR(50)
24 );
25
26 CREATE TABLE Incident (
27   Incident_ID INT PRIMARY KEY,
28   Asset_ID INT,
29   Analyst_ID INT,
30   Incident_Type VARCHAR(100),
31   Severity VARCHAR(20),
32   Status VARCHAR(50),
33   Detection_Date DATE,
34   FOREIGN KEY (Asset_ID) REFERENCES Asset(Asset_ID),
35   FOREIGN KEY (Analyst_ID) REFERENCES Analyst(Analyst_ID)
36 );
37
```

Output:

Tables_in_sandbox_db
alert
analyst
asset
incident
organization
remediation
report
vulnerability

Figure 3: Output of Data Definition Language (DDL) Execution Showing Created Tables

DML (Insert Data)

INSERT INTO Organization VALUES

(1, 'SecureNet Ltd', 'Finance', 'admin@securenet.com');

SELECT * FROM Organization;

INSERT INTO Asset VALUES

(101, 1, 'Main Firewall', 'Firewall', '192.168.1.1', 'Cisco IOS'),

(102, 1, 'Web Server', 'Server', '192.168.1.10', 'Ubuntu Linux');

SELECT * FROM Asset;

INSERT INTO Analyst VALUES

(201, 'Robert Iroha', 'robert@securenet.com', 'CCNA CyberOps', 'SOC Analyst');

```

SELECT * FROM Analyst;

INSERT INTO Incident VALUES
(301, 102, 201, 'SQL Injection Attempt', 'High', 'Open', '2026-05-20');

SELECT * FROM Incident;

INSERT INTO Alert VALUES
(401, 301, 'SIEM', 'Critical', '2026-05-20 14:30:00');

SELECT * FROM Alert;

INSERT INTO Vulnerability VALUES
(501, 102, 'CVE-2025-1234', 'High', '2026-05-18');

SELECT * FROM Vulnerability;

INSERT INTO Remediation VALUES
(601, 301, 'Blocked malicious IP and patched server', '2026-05-21', 'Completed');

SELECT * FROM Remediation;

INSERT INTO Report VALUES
(701, 301, 'Incident successfully contained and mitigated.', '2026-05-22');

SELECT * FROM Report;

```

Queries + Output

Query 1 — View All Incidents:

```
SELECT * FROM Incident;
```

Expected output

Incident_ID	Asset_ID	Analyst_ID	Incident_Type	Severity	Status
301	102	201	SQL Injection Attempt	High	Open

The screenshot shows a MySQL query editor with the following SQL code:

```

82 (102, 1, 'Web Server', 'Server', '192.168.1.10', 'Ubuntu Linux');
83
84
85
86
87 INSERT INTO Analyst VALUES
88 (201, 'Robert Iroha', 'robert@securenet.com', 'CCNA CyberOps', 'SOC Analyst');
89
90
91
92
93 INSERT INTO Incident VALUES
94 (301, 102, 201, 'SQL Injection Attempt', 'High', 'Open', '2026-05-20');
95
96
97
98
99 INSERT INTO Alert VALUES
100 (401, 301, 'SIEM', 'Critical', '2026-05-20 14:30:00');
101
102
103
104
105 INSERT INTO Vulnerability VALUES
106 (501, 102, 'CVE-2025-1234', 'High', '2026-05-18');
107
108
109
110
111 INSERT INTO Remediation VALUES
112 (601, 301, 'Blocked malicious IP and patched server', '2026-05-21', 'Completed');
113
114
115
116
117 INSERT INTO Report VALUES
118 (701, 301, 'Incident successfully contained and mitigated.', '2026-05-22');
119
120 SELECT * FROM Incident;

```

The output window shows the following table:

Incident_ID	Asset_ID	Analyst_ID	Incident_Type	Severity	Status	Detection_Date
301	102	201	SQL Injection Attempt	High	Open	2026-05-20

Query 2 — Show Incidents with Assigned Analysts:

SELECT

Incident.Incident_ID,

Incident.Incident_Type,

Analyst.Full_Name

FROM Incident

JOIN Analyst

ON Incident.Analyst_ID = Analyst.Analyst_ID;

Expected output

Incident_ID	Incident_Type	Full_Name
301	SQL Injection Attempt	Robert Iroha

queries.sql × +

44qhv7bhd

AI MySQL RUN

```

88 (201, 'Robert Iroha', 'robert@securenet.com', 'CCNA CyberOps', 'SOC Analyst');
89
90
91
92
93 INSERT INTO Incident VALUES
94 (301, 102, 201, 'SQL Injection Attempt', 'High', 'Open', '2026-05-20');
95
96
97
98
99 INSERT INTO Alert VALUES
100 (401, 301, 'SIEM', 'Critical', '2026-05-20 14:30:00');
101
102
103
104
105 INSERT INTO Vulnerability VALUES
106 (501, 102, 'CVE-2025-1234', 'High', '2026-05-18');
107
108
109
110
111 INSERT INTO Remediation VALUES
112 (601, 301, 'Blocked malicious IP and patched server', '2026-05-21', 'Completed');
113
114
115
116
117 INSERT INTO Report VALUES
118 (701, 301, 'Incident successfully contained and mitigated.', '2026-05-22');
119
120 SELECT
121     Incident.Incident_ID,
122     Incident.Incident_Type,
123     Analyst.Full_Name
124 FROM Incident
125 JOIN Analyst
126 ON Incident.Analyst_ID = Analyst.Analyst_ID;
127

```

Input for the program (Optional)

Output:

Incident_ID	Incident_Type	Full_Name
301	SQL Injection Attempt	Robert Iroha

Query 3 — List Vulnerabilities:

```
SELECT Asset.Asset_Name, Vulnerability.CVE_Code, Vulnerability.Severity
FROM Vulnerability
JOIN Asset
ON Vulnerability.Asset_ID = Asset.Asset_ID;
```

Expected output

Asset_Name	CVE_Code	Severity
Web Server	CVE-2025-1234	High

queries.sql44qhv7bhdMySQLAI RUN

```
84
85
86
87 INSERT INTO Analyst VALUES
88 (301, 'Robert Iroha', 'robert@securenet.com', 'CCNA CyberOps', 'SOC Analyst');
89
90
91
92
93 INSERT INTO Incident VALUES
94 (301, 102, 201, 'SQL Injection Attempt', 'High', 'Open', '2026-05-20');
95
96
97
98
99 INSERT INTO Alert VALUES
100 (401, 301, 'SIEM', 'Critical', '2026-05-20 14:30:00');
101
102
103
104
105 INSERT INTO Vulnerability VALUES
106 (501, 102, 'CVE-2025-1234', 'High', '2026-05-18');
107
108
109
110
111 INSERT INTO Remediation VALUES
112 (601, 301, 'Blocked malicious IP and patched server', '2026-05-21', 'Completed');
113
114
115
116
117 INSERT INTO Report VALUES
118 (701, 301, 'Incident successfully contained and mitigated.', '2026-05-22');
119
120 SELECT Asset.Asset_Name, Vulnerability.CVE_Code, Vulnerability.Severity
121 FROM Vulnerability
122 JOIN Asset
123 ON Vulnerability.Asset_ID = Asset.Asset_ID;
```

Input for the program (Optional)

Output:

Asset_Name	CVE_Code	Severity
Web Server	CVE-2025-1234	High

Conclusion

This project designed and implemented a cybersecurity incident response and threat monitoring database system for a SOC environment. The system supports asset tracking, incident management, vulnerability monitoring, and security reporting using a relational database structure. The implementation demonstrates how database systems can improve cybersecurity operations and incident response efficiency.